Constructor, operator "new"

03 November 2026 18:41

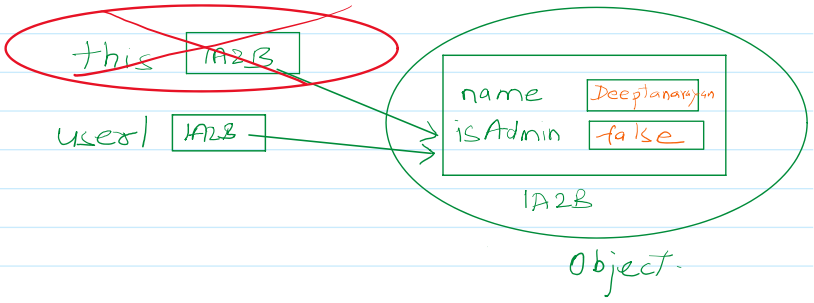
The regular {…} syntax allow us to create on object. But often we need to create many similar objects, like multiple user or menu items and so on.

That can be done using constructor functions and the "new" operator.

Constructor function

Constructor functions technically are regular functions. There are two conventions though:
1. They are named with capital letter first.
2. They should be executed only with "**new**" operator.

For instance:




```
function User(name){
    this.name = name;
    this.isAdmin = false;
}
let user1 = new User("Deeptanarayan");
alert(user1.name); //Deeptanarayan
alert(user1.isAdmin); // false
```

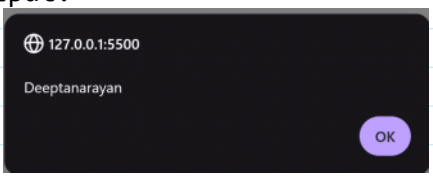
content of 'this'

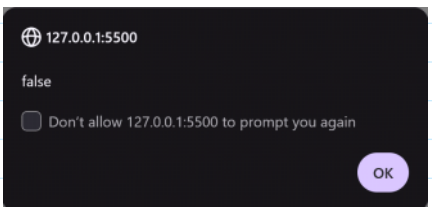
When a function is executed with new, it does the following steps:
1. A new empty object is created and assigned to **this**.
2. The function body executes. Usually it modifies **this** adds new properties to it.
3. The value of **this** is returned.

In other words, **new** user (...) does something like:

```
function User(name){
    // this = {}; (implicitly)
    this.name = name;
    this.isAdmin = false;
    //return this; (implicitly)
}
```

Output:





Constructor mode test: new.target
Inside a function we can check whether it was called with new or without it.
Using a special **new.target** property.

It is undefined for regular calls and equals the function if called with new:

```
function User(){
    console.log(new.target);
}
// without "new":
User(); //undefined
//with "new":
new User(); // function User(){...}
```

That can be used inside the function to know whether it was called with **new** "in constructor mode" or without it "in regular mode".

```
function User(name){
    if(!new.target){ //if you run me without new
        return new User(name);  // I will add new for you
    }
    this.name = name;
}
let deep = User('Deeptanarayan'); // redirects call to new user
console.log(deep.name);
let subh = new User("Subhalakshmi");
console.log(subh.name)
```

Output:
```
Deeptanarayan
Subhalakshmi
```

This approach is sometime used in libraries to make the syntax more flexible. So that people may call the function with or without **new** and it still works.

Probably not a good thing to use everywhere though, because omitting **new** makes it a bit less obvious what's going on. With **new** we all know that the new object is being created.

Return from constructors
Usually constructors do not have a **return** statement. Their task is to write all necessary stuff into **this** and it automatically becomes the result.

But if there is a return statement then the rule is simple:
- If **return** is called with an object, then the object is returned instead of **this**.
- If **return** is called with a primitive, its ignored.

In other words, **return** with an object returns the object in all other cases **this** is returned.

For instance here **return** overrides **this** by returning an object.

Code:

```
function BigUser(name){
    this.name = name;
    return {name: 'Godzilla' }; //<-- returns this object
}
console.log(new BigUser('John'));
console.log(new BigUser('James'));
console.log(new BigUser('Ann'));
```

Output:

```
{ name: 'Godzilla' }
{ name: 'Godzilla' }
{ name: 'Godzilla' }
```

Methods in Constructor

Using constructor functions to create objects gives a great deal of flexibility. The constructor function may have parameters that define how to construct the object and what to put in it.

Of course we can add to this not only properties but methods as well.

For instance, **new User(name)** below creates an object with the given name and the method **sayHi**:

```
function User(name){
    this.name = name;
    this.sayHi = function(){
        console.log("My name is: "+this.name);
    };
}
let deep = new User("Deeptanarayan");
deep.sayHi();
```

Output:

```
My name is: Deeptanarayan
```

Summary:

Constructor functions are. Briefly, constructors are regular functions, but there's a common agreement to the name with a capital letter first.
Constructor function should only be called using **new**. Such a call implies a creation of empty **this** at the start and returning the populated one at the end.

We can use constructor function to make multiple similar objects.

Javascript provides constructor function for many built-in language object: like Date for dates, Set for sets and others.